

# REFERRING MULTI-OBJECT TRACKING IN MOVING-CAMERA VIDEOS VIA GLOBAL MOTION COMPENSATION

*Author(s) Name(s)*

Author Affiliation(s)

## ABSTRACT

Referring multi-object tracking (RMOT) takes a video and a language expression as input and tracks all referred objects. Many tracking requirements involve how an object moves rather than how it appears. However, in a video captured by a moving camera, a parked vehicle may appear to move, while a moving vehicle may show little displacement. Existing RMOT methods relate motion with text but do not explicitly remove camera-induced motion. In this paper, we propose extracting residual motion across frames by estimating camera motion in driver-view videos and compare motion characteristics with the query expression. We consider the motion-matching extent and integrate it with the RMOT method’s prediction result through late fusion. In the evaluation, we verify the performance gain of taking the motion compensation module as a plug-in across different RMOT hosts.

**Index Terms**— referring multi-object tracking, motion compensation, motion-language alignment

## 1. INTRODUCTION

Referring multi-object tracking (RMOT) outputs tracking results of the objects described by a free-form language expression. For example, a “left-vehicles-in-red” description targets the red vehicles on the left. However, many descriptions focus on moving objects, such as “moving cars” or “a vehicle turning left”. In videos captured by a moving camera, the observed motion is a mixture of object motion and camera motion; thus, a static car may appear to move, while a moving car may appear static across frames.

Previous RMOT methods [1, 2, 3] match language expressions to objects. VMRMOT [4] considers explicit motion cues, such as changes in direction and speed, in the matching process. These methods compute motion from the object’s bounding-box displacement across frames, which mixes object motion with camera motion. On the other hand, conventional multi-object tracking (MOT) methods propose to remove camera motion. BoT-SORT [5] estimates camera-induced image motion, while UCMCTrack [6] stabilizes object trajectories. Overall, how motion compensation can be extended to RMOT is still underexplored.

In this paper, we propose separating object motion from camera motion and providing motion cues for RMOT methods. It consists of four stages. First, the geometric transformation across frames is estimated to measure camera motion. The estimated transformations are accumulated across frames to reliably measure object motion. Second, residual velocity, which represents object motion after removing camera motion, is computed across frames and used as a temporal motion feature. Third, a Contrastive Motion Aligner is developed to compare motion features with language expressions and produce a motion-matching score. Finally, the motion-matching score is scaled by a host-specific weight and added to the host’s matching score.

Here, the host model refers to a two-stage RMOT method used as a baseline, such as iKUN [2] and FlexHook [3]. The proposed process is implemented as a plug-in module, thereby improving existing RMOT baselines without retraining them.

This work connects motion compensation with motion-language modeling for RMOT. Our main contributions are as follows.

- We propose a plug-in module that aligns the object’s residual velocity with the referring expression, thereby improving the host RMOT method without modifying or retraining it.
- Across two host RMOT methods, i.e., iKUN and FlexHook, the proposed module improves pooled HOTA in all settings. On iKUN, it raises moving-class HOTA from 27.697 to  $37.139 \pm 0.923$  (approximately +9.44 points).

## 2. RELATED WORKS

### 2.1. Referring Multi-Object Tracking

TransRMOT [1] introduced the RMOT task on the Refer-KITTI dataset. Later methods improved different parts of this task. iKUN [2] separated language-based object matching from object tracking, while FlexHook [3] improved how visual and language information interact. ReferGPT [7] used a multimodal large language model to describe existing object tracks, and MEX [8] provided a memory-efficient module for matching tracks with language.

Recent RMOT methods [9] further improve models’ understanding of object descriptions. STORM [10] unifies grounding and tracking in an end-to-end multimodal large language model. VM-RMOT [4] explicitly represents motion using information such as position, direction, changes in distance, and changes in speed. These methods estimate object motion from the object’s bounding-box movements across frames.

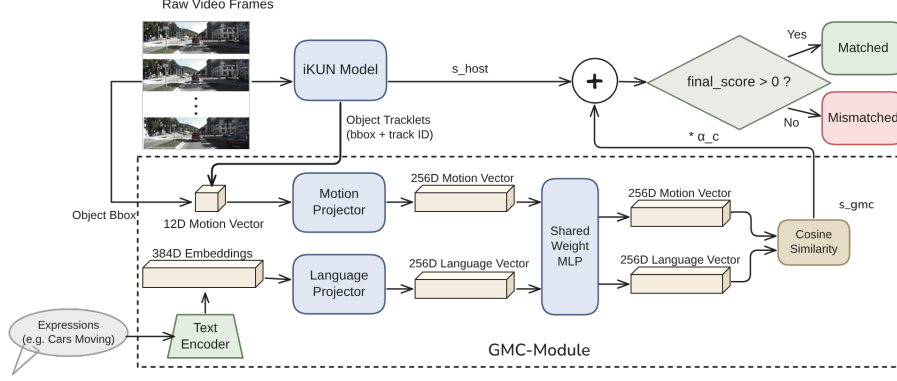
### 2.2. Motion Compensation in Tracking

Camera motion compensation is widely used in traditional multi-object tracking. BoT-SORT [5] estimates camera-induced motion across frames. UCMCTrack [6] estimates how the camera moves relative to the road. EMAP [11] further separates the camera’s turning and movement from object motion within the Kalman prediction process.

These methods demonstrate that compensating for camera motion can improve tracking. However, how to integrate compensated motion cues with language expressions to improve RMOT remains underexplored.

## 3. METHOD

Figure 1 shows the proposed RMOT framework with the global motion compensation (GMC) module. The GMC module is attached to



**Fig. 1.** The proposed RMOT framework with the global motion compensation module. The host model (iKUN in this case) predicts an object-language matching score. The GMC module provides an extra matching score to adjust matching results.

a two-stage RMOT method, iKUN [2] in this case. iKUN performs object detection and tracking and then outputs object-language matching scores. The inputs to the GMC module are the sequence of object bounding boxes across frames, the video frames, and the language expression. Based on these inputs, it estimates camera motion, extracts residual object motion, aligns it with the language expression, and produces another matching score. The score is then combined with the host model’s predicted score to determine the final tracking result. The GMC module consists of four stages: motion compensation, multi-scale residual velocity extraction, motion-language alignment, and late fusion.

### 3.1. Motion Compensation

As in previous RMOT works, we take Refer-KITTI [1] as the evaluation benchmark. Refer-KITTI is recorded from a driver-view camera, so the road surface is visible in most frames and stays close to a single plane. For a forward-facing camera the horizon lies near the vertical image center, so the lower half of the frame contains the road surface. We extract Shi-Tomasi keypoints [12] in this lower half, excluding the interiors of the tracked object boxes, and track them into the next frame with pyramidal Lucas-Kanade optical flow [13]. The RANSAC [14] algorithm is then employed to estimate the frame-to-frame homography  $H_{t-1 \rightarrow t}$  that describes the correspondences between keypoints in two frames.

To describe camera motion across more frames, the estimated homographies are accumulated as

$$H_{t-k \rightarrow t} = H_{t-1 \rightarrow t} \times H_{t-2 \rightarrow t-1} \times \cdots \times H_{t-k \rightarrow t-k+1}, \quad (1)$$

where  $H_{t-k \rightarrow t}$  represents the accumulated camera transformation from frame  $I_{t-k}$  to frame  $I_t$ . Instead of repeatedly transforming already-warped coordinates, the accumulated homography is applied only once to the detected objects’ centroids. This reduces numerical drift and provides a stable reference for estimating the residual velocity in the next stage.

### 3.2. Multi-Scale Residual Velocity

Before going into more details, we define the following three velocities:

- **Raw velocity:** Raw velocity is the observed image-plane velocity of an object, computed from the displacement of its

centroid between two frames separated by  $g$  frames and normalized by the corresponding time interval.

- **Ego-motion-induced velocity:** Ego-motion-induced velocity is the image-plane velocity caused by camera motion, computed from the displacement of the object’s location predicted by the homography between two frames separated by  $g$  frames and normalized by the corresponding time interval.
- **Residual velocity:** Residual velocity is the camera-motion-compensated object velocity obtained by subtracting the ego-motion-induced velocity from the raw velocity.

Based on the accumulated homography matrix and the objects’ bounding boxes, we aim to estimate the *residual velocity*. For a temporal gap  $g$ , the raw velocity  $v_g^{\text{raw}}$  is calculated by dividing the displacement of the object centroid between two frames ( $I_{t-g}$  and  $I_t$ ) by the time gap  $g$ . The raw velocity is mixed with the object’s motion and the camera’s motion. To remove the camera motion, the accumulated homography is applied to an object’s centroid  $\mathbf{o}_{t-g}$ , yielding  $\hat{\mathbf{o}}_t$  that is the predicted centroid at  $I_t$  under the assumption that the object is static. Then the ego-motion-induced displacement  $\hat{\mathbf{o}}_t - \mathbf{o}_{t-g}$  is normalized by the time interval  $g$  to estimate the ego-motion-induced velocity  $v_g^{\text{ego}}$ . The residual velocity  $v_g^{\text{res}}$  is then computed as

$$v_g^{\text{res}} = v_g^{\text{raw}} - v_g^{\text{ego}}. \quad (2)$$

This value approximates the motion that a static camera would observe. For example, a parked vehicle has near-zero residual velocity even when camera motion causes a large image displacement. All displacements are normalized by the image dimensions to make them independent of the image resolution. In the implementation the factor  $1/g$  is omitted; this is equivalent up to a per-gap rescaling of the first projection layer of Sec. 3.3.

To capture motion over different time spans, residual velocities are computed at temporal gaps of  $\{2, 5, 10\}$  frames. That means the accumulated homography matrices mentioned in Eqn. (1) have three versions, i.e.,  $H_{t-2 \rightarrow t}$ ,  $H_{t-5 \rightarrow t}$ , and  $H_{t-10 \rightarrow t}$ . The residual velocities are concatenated with the object’s box information to form a 12-dimensional motion feature,

$$((dx_s, dy_s), (dx_m, dy_m), (dx_l, dy_l), \Delta w, \Delta h, c_x, c_y, w, h), \quad (3)$$

where the first six values are the residual velocities at the three temporal gaps, and  $\Delta w, \Delta h$  capture the change in box scale. The terms  $c_x$  and  $c_y$  describe the coordinates of the box center, and  $w$  and  $h$  describe the width and height of the box, respectively.

A track contributes a motion feature only once it has 11 contiguous frames of history, the shortest history at which all three gaps are defined; on shorter histories the module abstains and the host score passes through unchanged. The threshold follows from the largest gap and adds no free parameter.

### 3.3. Motion-Language Matching

Given the motion features mentioned above and the language expression, this stage measures how much the observed object’s motion matches the expression. The 12-dimensional motion feature is encoded by a motion encoder, while the language expression is encoded by the model called all-MiniLM-L6-v2 (MiniLM) [15] into the text embeddings. Both embeddings are projected into the same feature space using linear projection layers and a shared Multi-Layer Perceptron (MLP).

The model is trained using the Information Noise Contrastive Estimation (InfoNCE) [16] loss to maximize the similarity between matched motion-expression pairs while separating unmatched pairs. A positive pair is formed by the embeddings of an object and the corresponding expression. Other object-expression pairs in the same mini-batch are treated as negatives. The MiniLM text encoder is kept fixed, and only the projection layers and motion encoder are trained. During inference, the cosine similarity  $s_{\text{gmc}}$  between an object’s motion embeddings and the target expression’s embeddings is used as the matching score and passed to the late fusion stage described below.

### 3.4. Fusion

We aim to integrate the matching score described above with the prediction score from the host RMOT method to improve tracking performance. The integration is defined as

$$s_{\text{final}} = s_{\text{host}} + \alpha_c s_{\text{gmc}}, \quad (4)$$

where  $s_{\text{host}}$  is the prediction score from the host model. The weighting  $\alpha_c$  is designed to vary for different types of expressions. For example, “moving car” is a *moving* expression, “left vehicles which are parking”<sup>1</sup> is a *static* expression, and “right cars in light color” is an *appearance* expression. Based on the keywords in the expression, we classify it as moving, static, or appearance. Because different host models produce scores over different ranges, the weighting  $\alpha_c$  is selected for each host via leave-one-sequence-out cross-validation. For each fold, the weighting that yields the highest pooled HOTA on all but one sequence is chosen. The final weighting is determined as the median over folds. Finally, when iKUN is used as the host model, the weighting  $\alpha_c$  is set to 1.0 for moving and static expressions, while the weighting  $\alpha_c$  is set to 0.1 for the appearance expressions. When FlexHook V1 is used as the host model, the weighting  $\alpha_c$  is set to 7 for all expressions. When FlexHook V2 is used as the host model, the weighting  $\alpha_c$  is set to 5 for all expressions.

Since the GMC module only adds extra influence on the final prediction score, it can be integrated into existing two-stage RMOT frameworks without changing or retraining their detector, tracker, visual encoder, or language encoder.

**Table 1.** Pooled HOTA on the Refer-KITTI test sets (mean  $\pm$  std over three seeds). The DetA and AssA cells give the reproduced host value and the value with the GMC module.

| Host            | w/o GMC | w. GMC             | DetA        | AssA        |
|-----------------|---------|--------------------|-------------|-------------|
| iKUN [2]        | 44.56   | 45.304 $\pm$ 0.115 | 32.04→32.90 | 62.48→62.94 |
| FlexHook V1 [3] | 53.824  | 53.980 $\pm$ 0.059 | 43.35→43.51 | 66.92→67.07 |
| FlexHook V2 [3] | 42.526  | 42.625 $\pm$ 0.032 | 30.63→30.75 | 59.19→59.23 |

## 4. EXPERIMENTS

### 4.1. Setup

We evaluate the proposed module on the Refer-KITTI V1 [1] and the Refer-KITTI V2 [17] benchmarks using the HOTA metric from TrackEval [18], which jointly measures detection and association performance. We report both moving-class HOTA, which evaluates only the moving-class expressions (those assigned to the moving class by the keyword classifier of Sec. 3.4), and pooled HOTA, which evaluates all expressions in the benchmark. The V1 test split contains three sequences, and the V2 test split contains four.

The proposed GMC module is integrated with two host models, i.e., iKUN [2] and FlexHook [3], respectively. For comparison, we also evaluate a raw-velocity variant that uses image-plane motion directly, without ego-motion compensation. The GMC module is trained using the AdamW [19] optimizer with a learning rate of  $10^{-3}$ , batch size 256, and 100 epochs. The linear projectors and MLP in the module are trained based on the Refer-KITTI V1 training split (15 sequences, disjoint from the test sequences). Each positive pair consists of an expression and the motion features derived from the object’s ground-truth boxes. At test time, raw object movement is derived from the host model’s initial tracking results, the residual velocity is calculated, and the motion features are finally computed. The road-plane homography tracks up to 600 corners with a 3-pixel RANSAC threshold and succeeds on all 7,690 adjacent frame pairs of the 19 training and evaluation sequences. Excluding host inference, the module runs at 149 FPS on CPU (6.7 ms per frame).

Results are averaged over three random seeds for the main configurations. Ablation analyses are done using five seeds together with Welch’s *t*-test. Both host models use their original detection results, and every Refer-KITTI V1 setting is evaluated on the benchmark’s official 150-expression test list. Our reproduced FlexHook baselines match their published scores exactly at  $\alpha_c=0$ , while our reproduced iKUN (44.543) is marginally below its published score (44.56); Table 1 therefore reports the stricter comparison against the published baselines.

Refer-KITTI does not provide a dedicated validation split. All fusion weights are therefore selected by the leave-one-sequence-out procedure mentioned in Sec. 3.4. No weight is chosen on the sequence it is evaluated on.

Refer-KITTI is highly imbalanced: only 21 of the 150 V1 and 111 of the 862 V2 test expressions are moving-class expressions. Consequently, improvements in motion understanding may have only a limited impact on pooled HOTA. Therefore, moving-class HOTA is used as the primary evaluation metric, while pooled HOTA is reported to reflect overall benchmark performance.

<sup>1</sup>in Refer-KITTI, this expression refers to a vehicle that is already parked, rather than a vehicle that is in the process of parking.

**Table 2.** Moving-class HOTA (moving-class expressions: 21/150 for iKUN and FlexHook V1, 111/862 for FlexHook V2),  $n=3$  seeds.

| Host        | Baseline | Performance gain |
|-------------|----------|------------------|
| iKUN        | 27.70    | $+9.44 \pm 0.92$ |
| FlexHook V1 | 47.90    | $+0.43 \pm 0.19$ |
| FlexHook V2 | 38.56    | $+0.69 \pm 0.04$ |

**Table 3.** Per-class HOTA ablation on iKUN ( $n=5$ ); single  $\alpha$ :  $\alpha_{\text{mot}}=\alpha_{\text{app}}=0.35$  (LOSO). Others = all expressions outside the moving class.

| Variant                      | Moving HOTA                        | Others HOTA                        | Pooled HOTA                        |
|------------------------------|------------------------------------|------------------------------------|------------------------------------|
| Native host (w/o GMC)        | 27.70                              | 45.87                              | 44.54                              |
| Full module (w. GMC)         | <b><math>36.99 \pm 0.68</math></b> | <b><math>46.05 \pm 0.05</math></b> | <b><math>45.28 \pm 0.09</math></b> |
| single $\alpha$ (no routing) | $32.42 \pm 0.25$                   | $45.99 \pm 0.12$                   | $44.95 \pm 0.11$                   |
| – ego (raw velocity only)    | $32.37 \pm 0.39$                   | $45.77 \pm 0.04$                   | $44.61 \pm 0.07$                   |
| – multiscale (single gap)    | $35.14 \pm 0.38$                   | $45.96 \pm 0.04$                   | $45.00 \pm 0.02$                   |

## 4.2. Main results

Table 1 summarizes the pooled HOTA results. We evaluate the performance variations when iKUN or FlexHook is used as the host model, with or without the GMC module. Specifically, the values for the rows of iKUN and FlexHook V1 are obtained from the Refer-KITTI V1 dataset, while the values for the row of FlexHook V2 are obtained from the Refer-KITTI V2 dataset. Table 1 shows that the proposed GMC module provides performance gains across all settings; all three settings exceed the hosts’ published pooled HOTA ( $+0.744$ ,  $+0.156$ , and  $+0.099$ , respectively).

Table 2 reports HOTA variations on the moving-class expressions only. As can be seen, the GMC module provides performance improvement across all three settings. The performance gains differ by an order of magnitude: iKUN gains  $+9.44 \pm 0.92$ , FlexHook V1  $+0.43 \pm 0.19$ , and FlexHook V2  $+0.69 \pm 0.04$ .

## 4.3. Ablation Studies

Table 3 reports the five-seed ablation measurements when iKUN is used as the host model. The full module improves moving-class HOTA from 27.70 to  $36.99 \pm 0.68$  and pooled HOTA from 44.54 to  $45.28 \pm 0.09$ . If the ego-motion compensation is removed, the moving-class HOTA drops from 36.99 to 32.37, while removing the multi-scale gaps when calculating residual velocity reduces it from 36.99 to 35.14. Both drops are significant under Welch’s  $t$ -test ( $n=5$ ; ego:  $t=13.1$  moving-class, 13.2 pooled; multi-scale:  $t=5.3$  and 6.8; all  $p<0.01$ ). The expressions outside the moving class change little across the variants (Others in Table 3). These show the influence of both components.

Figure 2 shows a sample tracking result. The parked car appears to move across frames due to camera motion, while the moving car remains at nearly the same distance from the camera. Without ego-motion compensation, these two cases are easily confused and mistracked. After motion compensation, the parked car has nearly zero residual velocity, while the moving car presents nonzero residual velocity. This allows the GMC module to more correctly predict the matching score.

## 5. CONCLUSION AND LIMITATIONS

The GMC module is a plug-in that improves motion grounding in referring multi-object tracking without modifying the detector, tracker,



**Fig. 2.** “Moving cars” (Refer-KITTI seq 0005). The parked car (red) sweeps across the image as the camera passes, while the moving car (green) barely shifts. Raw image motion misclassifies both; the ego-compensated residual recovers both from the same host score.

or appearance-language matching model. By compensating for camera motion and matching residual motion features with language expressions, the proposed module helps distinguish object motion from camera-induced motion. Road-region background correspondences provide a geometric reference for estimating camera motion in driving videos. The residual motion features are then matched against language expressions to yield a matching score, which is integrated with the host model’s prediction score via late fusion. On the Refer-KITTI datasets, the pooled HOTA improves for different host models. On the moving-class expressions, the largest gain is observed when iKUN is used as the host model ( $+9.44 \pm 0.92$ ).

Despite these promising results, several limitations remain. First, the road-plane homography assumes a flat ground plane. Scenes with steep slopes or uneven terrain fall outside the model’s scope. Second, the keyword classifier routes only moving-class expressions to  $\alpha_{\text{mot}}$ ; direction and turning expressions (214 of the 818 in Refer-KITTI V1) receive the appearance weight, so the module does not help them even though they describe motion. Finally, our experiments are conducted on two RMOT frameworks and the Refer-KITTI benchmarks. Additional evaluations across more host models and datasets are needed to further validate the generality of the proposed approach.

## 6. REFERENCES

- [1] Dongming Wu, Wencheng Han, Tiancai Wang, Xingping Dong, Xiangyu Zhang, and Jianbing Shen, “Referring multi-object tracking,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023, pp. 14633–14642, IEEE.
- [2] Yunhao Du, Cheng Lei, Zhicheng Zhao, and Fei Su, “iKUN: Speak to trackers without retraining,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024, pp. 19135–19144, IEEE.

- [3] Weize Li, Yunhao Du, Qixiang Yin, Zhicheng Zhao, and Fei Su, “Rethinking two-stage referring-by-tracking in referring multi-object tracking: Make it strong again,” 2025, Accepted to CVPR 2026.
- [4] Weiyi Lv, Ning Zhang, Hanyang Sun, Haoran Jiang, Kai Zhao, Jing Xiao, and Dan Zeng, “Vision–motion–reference alignment for referring multi-object tracking via multi-modal large language models,” 2025.
- [5] Nir Aharon, Roy Orfaig, and Ben-Zion Bobrovsky, “BoT-SORT: Robust associations multi-pedestrian tracking,” 2022.
- [6] Kefu Yi, Kai Luo, Xiaolei Luo, Jiangui Huang, Hao Wu, Rongdong Hu, and Wei Hao, “UCMCTrack: Multi-object tracking with uniform camera motion compensation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*. 2024, vol. 38, pp. 6702–6710, AAAI Press.
- [7] Tzoulio Chamiti, Leandro Di Bella, Adrian Munteanu, and Nikos Deligiannis, “Refergpt: Towards zero-shot referring multi-object tracking,” 2025, CVPR Workshops.
- [8] Huu-Thien Tran, Phuoc-Sang Pham, Thai-Son Tran, and Khoa Luu, “Mex: Memory-efficient approach to referring multi-object tracking,” 2025.
- [9] Wenyan He, Yajun Jian, Yang Lu, and Hanzi Wang, “Visual-linguistic representation learning with deep cross-modality fusion for referring multi-object tracking,” in *ICASSP 2024 – 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 6310–6314.
- [10] Zijia Lu, Jingru Yi, Jue Wang, Yuxiao Chen, Junwen Chen, Xinyu Li, and Davide Modolo, “STORM: End-to-end referring multi-object tracking in videos,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Findings*, 2026.
- [11] Navid Mahdian, Mohammad Jani, Amir M. Soufi Enayati, and Homayoun Najjaran, “Ego-motion aware target prediction module for robust multi-object tracking,” 2024.
- [12] Jianbo Shi and Carlo Tomasi, “Good features to track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1994, pp. 593–600.
- [13] Bruce D. Lucas and Takeo Kanade, “An iterative image registration technique with an application to stereo vision,” in *International Joint Conference on Artificial Intelligence (IJCAI)*, 1981, pp. 674–679.
- [14] Martin A. Fischler and Robert C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [15] Nils Reimers and Iryna Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992, Association for Computational Linguistics.
- [16] Aaron van den Oord, Yazhe Li, and Oriol Vinyals, “Representation learning with contrastive predictive coding,” 2018.
- [17] Yani Zhang, Dongming Wu, Wencheng Han, and Xingping Dong, “Bootstrapping referring multi-object tracking,” 2024, Introduces TempRMOT and Refer-KITTI-V2.
- [18] Jonathon Luiten, Aljoša Osěp, Patrick Dendorfer, Philip Torr, Andreas Geiger, Laura Leal-Taixé, and Bastian Leibe, “HOTA: A higher order metric for evaluating multi-object tracking,” *International Journal of Computer Vision*, vol. 129, no. 2, pp. 548–578, 2021.
- [19] Ilya Loshchilov and Frank Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.